

Контейнеры

ОСНОВЫ

Антон Титов

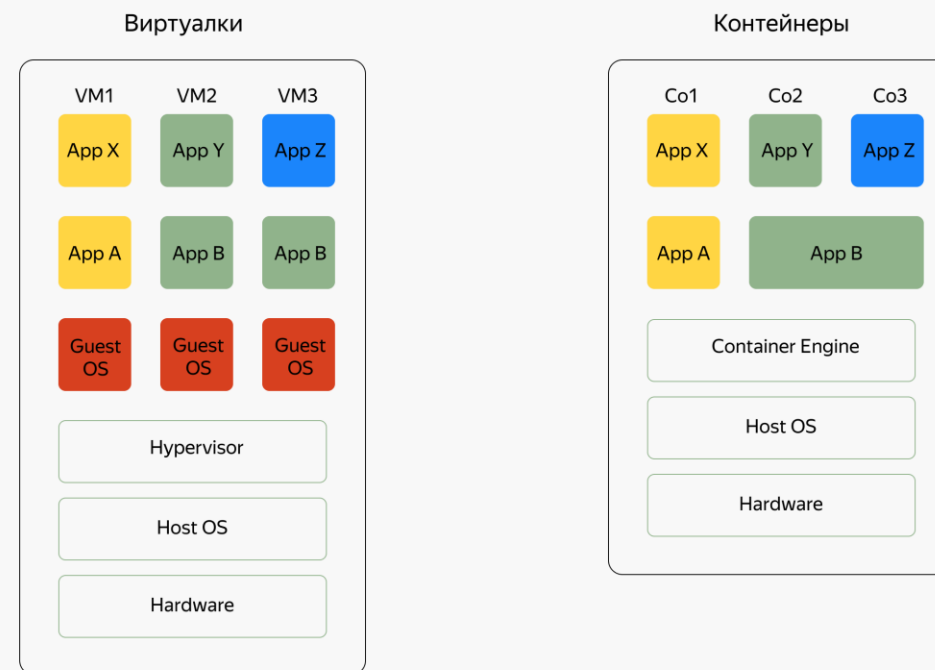
Руководитель группы защиты веб-приложений

Что такое Контейнеры?

Любая программа (Прикладное программное обеспечения, ППО) требует определенных функций ядра системы. Для простоты можно считать это зависимостями.

Виртуальные машины работают на гипервизоре, а он работает поверх хостовой ОС и эмулирует физические устройства. Из пункта выше получается, что для VM (как и физическим устройствам) нужна отдельная ОС.

Для контейнеров это не требуется.



Контейнеры – это легковесная сущность, процесс в системе, ограниченный и изолированный при помощи определенных механизмов.

Примеры использования контейнеров

Быстрое развертывание приложений (PostgreSQL)

Развертывание приложений, требующих разное окружение (изоляция)

Развертывание **не очень** безопасных приложений* (изоляция)

Лимитирование ресурсов для приложений (изоляция)

Симуляция и реализация как части, так и всей инфраструктуры

Заметили слово **изоляция**? Запомните

Средства управления контейнерами

Можно сделать три разделения

Runtimes

(низкоуровневые компоненты, которые непосредственно создают контейнеры)

- runc (low)
- crun (low)
- containerd (high)
- crio (high)

- docker
- podman

Контейнерные движки и CLI

(Обертки над рантаймами для упрощения взаимодействия с ними)

Оркестраторы контейнеров

(Обертки над рантаймами, работают как поверх движков, так и поверх рантаймов)

- k8s (kubernetes)
- OpenShift
- Docker Swarm

На низких уровнях существуют namespaces, cgroups, capabilities, и прочее. Это вы можете изучить самостоятельно

Отступление про namespace

Это всего лишь отступление

Namespace – функционал ядра Linux. Их всего 6:

1. PID namespace - сколько процессов я вижу
2. NET namespace - какая у меня сеть
3. MNT namespace - какой у меня диск
4. UTS namespace - как меня зовут
5. IPC namespace - с кем я могу общаться
6. USER namespace (часто, но не всегда) - кто я

```
[root@alt-1 ~]# ps aux | head -2
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.2 99728 10860 ?        Ss   мая21   13:53 /sbin/init splash
[root@alt-1 ~]# unshare --fork --pid --mount-proc bash
[root@alt-1 ~]# ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.1  0.1  7976  4556 pts/0    S    16:46   0:00 bash
root           4  0.0  0.0  6452   872 pts/0    R+   16:46   0:00 ps aux
```

Мы можем принудительно **ЗАПУСТИТЬ*** процесс в другом namespace. Пример: `unshare --fork --pid --mount-proc bash`

*При сдаче ДЗ можете приложить ресерч «почему **только запустить**» и получить доп. балл.

Контейнер – это когда процесс использует все **namespace** + **cgroups** + rootfs (mount + chroot + proc)

Отступление про cgroups

Это всего лишь отступление

Cgroups – функционал ядра Linux. Проще говоря, это структура в файловой системе `/etc/sys/cgroups`, строго определенного формата.

Мы можем создать нужную нам структуру и Linux сам выполнит лимитацию, **сколько ресурсов будет выделено** под процесс (группу процессов): CPU, RAM, и тд.

```
[root@alt-1 ~]# tree -L 2 /sys/fs/cgroup/system.slice/ssh.service/
/sys/fs/cgroup/system.slice/ssh.service/
├── cgroup.controllers
├── cgroup.events
├── cgroup.freeze
├── cgroup.kill
├── cgroup.max.depth
├── cgroup.max.descendants
├── cgroup.procs
├── cgroup.stat
├── cgroup.subtree_control
├── cgroup.threads
├── cgroup.type
├── cpu.stat
├── memory.current
├── memory.events
├── memory.events.local
├── memory.high
├── memory.low
├── memory.max
├── memory.min
├── memory.numa_stat
├── memory.oom.group
├── memory.peak
├── memory.reclaim
├── memory.stat
├── memory.swap.current
├── memory.swap.events
├── memory.swap.high
├── memory.swap.max
├── memory.zswap.current
├── memory.zswap.max
├── pids.current
├── pids.events
├── pids.max
└── pids.peak

0 directories, 34 files
[root@alt-1 ~]# cat /sys/fs/cgroup/system.slice/ssh.service/memory.max
max
```

Контейнер – это когда процесс использует все **namespace** + **cgroups** + rootfs (mount + chroot + proc)

Основные сущности Docker

Docker – один из движков управления контейнерами (иногда можно встретить, что это runtime, но всё таки это не так)

Рассмотрим основные сущности:

- Image
- Container
- Network
- Volume

Дополнительно рассмотрим:

- Logs
- Inspect
- Build
- Compose

Просто пример:

```
docker run -d --name test --network lab --volume app:/data debian:12 sleep infinity
```

Быстрый старт с Docker

Запустим простейший контейнер

Рассмотрим команду:

```
docker run debian echo "Hello World"
```

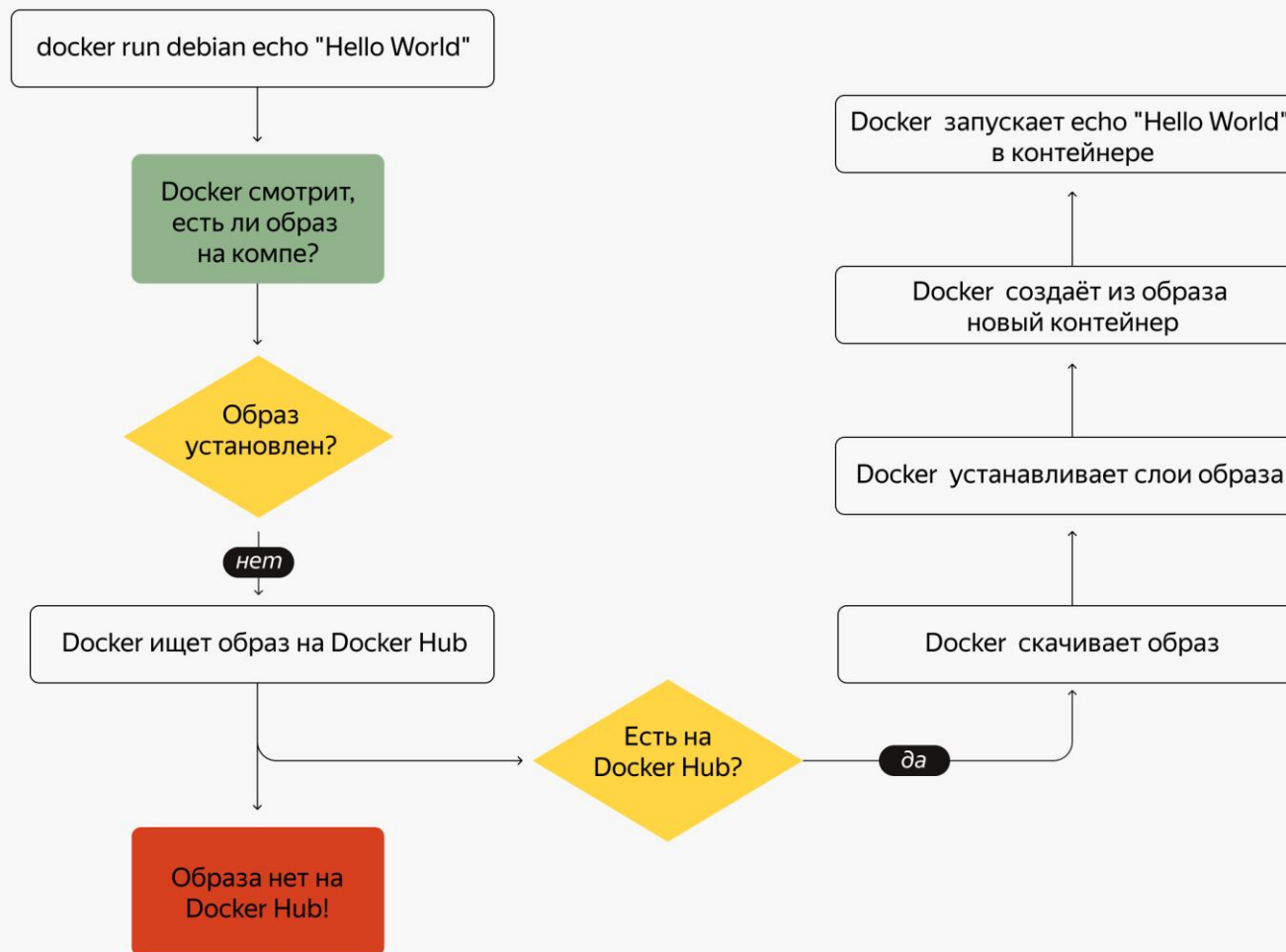
docker – утилита

run – команда на запуск контейнера

debian – образ для этого контейнера

echo "Hello World" – команда выполнения контейнера / главный процесс. Важно отметить:

1. Пишется всегда после образа
2. Когда команда завершится, контейнер остановится.



docker image

Image – это шаблон. На его основе будут создаваться контейнеры.

Шаблон мы можем создать самостоятельно, а можем скачать из репозитория. Самый популярный: Docker Hub.

Пример: `docker pull nginx:latest`

После выполнения команды будет загружен образ nginx последней версии



Команда: `docker image ls`

docker container



Контейнер — это обычный процесс , но у него есть три ключевых отличия:

1. изолированные процессы (он “видит” не всю систему)
2. своя файловая система (свой /)
3. своя сеть (свои интерфейсы и IP)

Это происходит за счёт связки **Namespace** + **cgroups** (минимально для контейнера, но недостаточно для docker-контейнера)

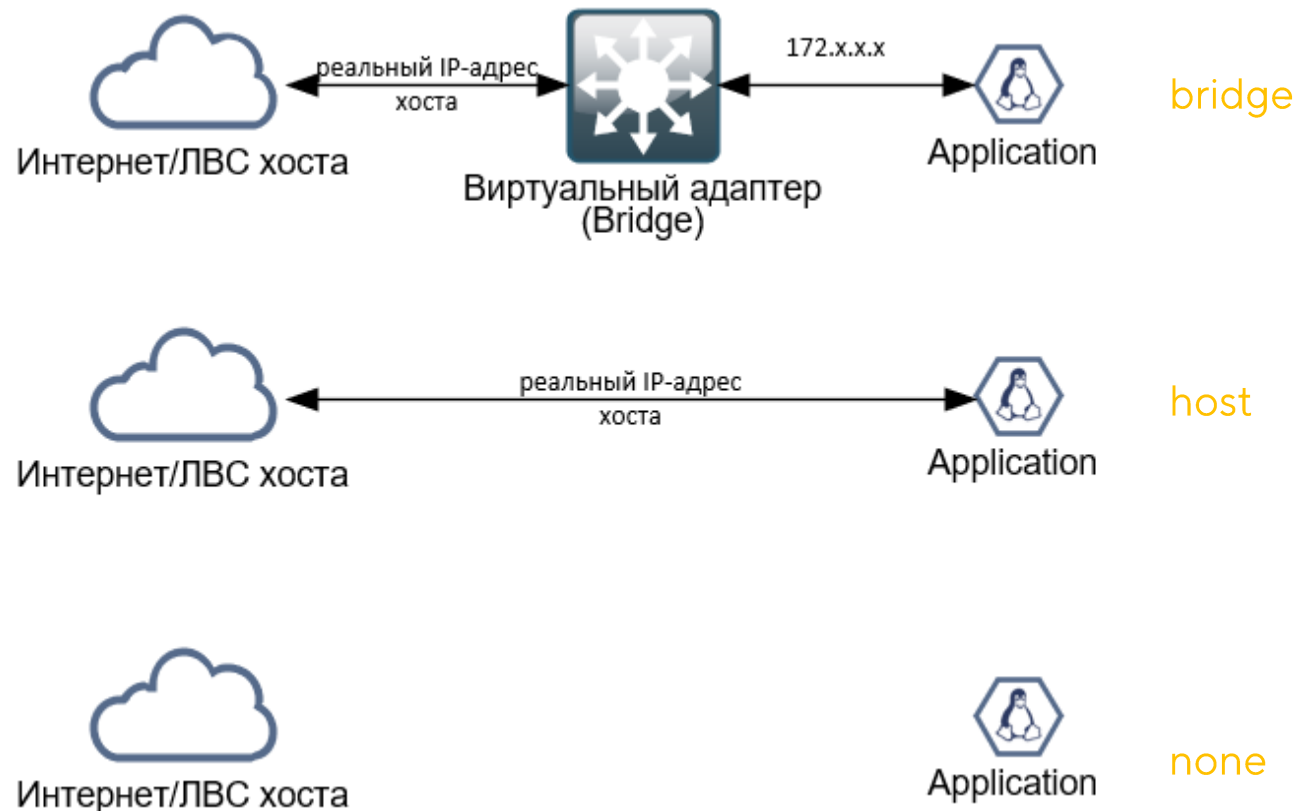
Докер работает на containerd

Команда: docker **container** ls –a (старомодный, но очень популярный вариант docker ps –a)

docker network

Контейнеры в докере можно связать с несколькими видами сетей (драйверами):

Тип	Плюсы	Минусы/ особенности
Bridge	1. Контейнеры могут общаться друг с другом 2. Можно публиковать порты (переопределять)	Снаружи контейнер не виден
Host	1. Нет NAT (не нужно публиковать порты) 2. Высокая производительность	Нет сетевой изоляции
None	Максимальная изоляция	Нет сети
Overlay	Контейнеры на разных машинах видят друг друга (VXLAN)	Используется при Docker Swarm
Другие	Редко встречаются	Сложно настраиваются



Команда: `docker network ls`

docker volume

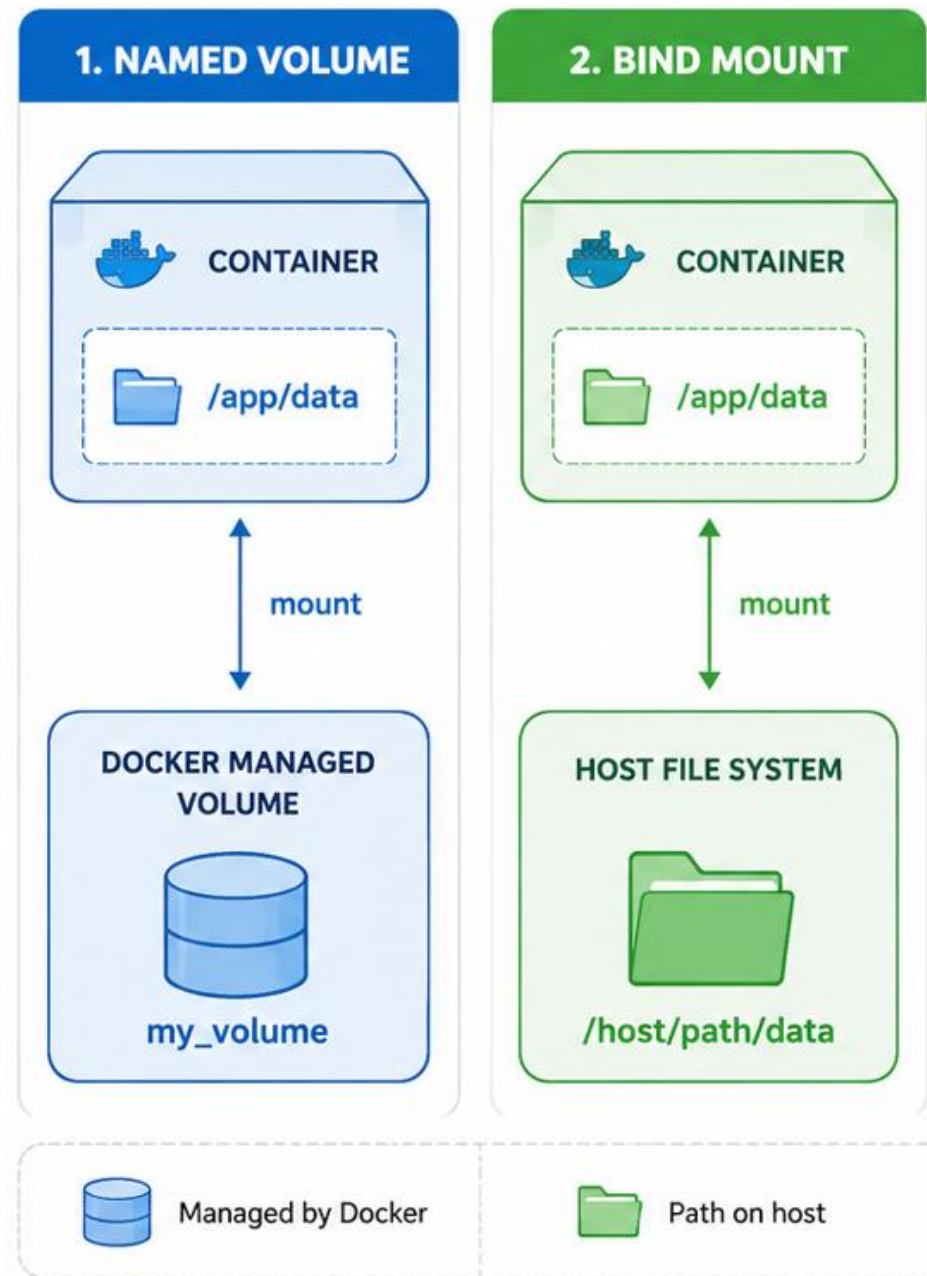
Все контейнеры являются **Stateless**

Это значит, что **при удалении** контейнера вы **потеряете все данные** в нём.

Решение – хранение данных вне контейнера.
Есть два основных способа это сделать:

Bind Mount – когда мы связываем директорию/файл в хостовой системе с чем-то внутри контейнера. Пример:
-v /host/path:/container/path

Named Volume – Docker сам управляет хранилищем. Пример:
-v my_volume:/container/path



Краткое подведение итогов

Быстро погрузиться в контейнеризацию поможет **Docker**

Контейнер – это легковесный изолированный* процесс

Для запуска контейнера потребуется образ

Скорее всего ваш контейнер должен общаться с сетью. Для этого подойдет Host или Bridge сети

По умолчанию контейнеры stateless.

Если нам нужен изменяемый конфиг, надо смотреть в сторону Bind Mounts и Named Volumes. Bind Mounts проще для дебага, Named Volumes – лучше для прода.

Запустить контейнер с автоматической подгрузкой образа в фоновом режиме:

```
docker run -d <image_name>
```

Создать отдельную подсеть:

```
docker network create <name> --driver <type>
```

Создать контейнер в подсети (после ее создания):

```
docker run -d --network <network_name> <image_name>
```

Создать контейнер с Named Volume:

```
docker run --volume <vol_name>:/path/to/dir <image_name>
```

Краткое подведение итогов

Рабочий пример

Создаем рабочую директорию

```
mkdir -p nginx
```

Формируем конфигурацию Nginx (EOF нужен для описания файлов с переносом строк, повторный EOF закроет ввод)

```
cat <<EOF > nginx/default.conf
```

```
server {  
    listen 80;  
    location / {  
        return 200 'Hello from containerized nginx';  
    }  
}  
EOF
```

Создаем собственную подсеть для Docker

```
docker network create my_custom_network
```

Запускаем сам контейнер

```
docker run -d --name nginx_test --network my_bridge_network \  
-p 8080:80 -v $(pwd)/nginx/default.conf:/etc/nginx/conf.d/default.conf:ro nginx:latest
```


docker logs

Иногда с контейнерами что-то случается.

Иногда они падают.

```
hapx@waf:~$ nano nginx.conf
hapx@waf:~$ docker run --name test -d -v ~/nginx.conf:/etc/nginx/nginx.conf nginx
36a46a4862a1149bcc44e353e89ebdcffad235863c1de8c9bc864acce0e38a45

hapx@waf:~$
hapx@waf:~$ docker ps -a | head -n 2
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS
36a46a4862a1   nginx     "/docker-entrypoint...." 32 seconds ago Exited (1) 3 seconds ago

hapx@waf:~$ docker logs test
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/06/11 10:44:24 [emerg] 1#1: invalid number of arguments in "worker_processes" directive in /etc/nginx/nginx.conf:3
nginx: [emerg] invalid number of arguments in "worker_processes" directive in /etc/nginx/nginx.conf:3
hapx@waf:~$ head -n 3 nginx.conf
user www-data;
worker_processes auto
pid /run/nginx.pid;
```

Команда: `docker logs <container_id OR container_name>`

docker inspect

Следующим уровнем траблшутинга является просмотр конфигураций контейнера.

Делается это следующим образом:

```
docker inspect [container_id] --format='{{json .NetworkSettings.Networks}}'
```

Полезные аргументы:

--format='{...}' позволяет уточнять место для поиска, например --
format='{{.NetworkSettings.Networks}}'

--format='{json ...}' аналогичная команда, но в отличие от первой будет выводить в формате json (первая выводит в standard python)

Еще круто использовать в связке утилиту **jq**.
Пример справа

```
{
  "Id": "9e1e1f2251252cf45734f00d4548b770d1f775a8e7aafb8e8b3c760d9923276",
  "Created": "2026-06-05T12:58:42.414089704Z",
  "Path": "/docker-entrypoint.sh",
  "Image": "sha256:e4720093a3c1381245b53a5a51b417963b3c4472d3f47fc301930a4f3",
  "ResolvConfPath": "/var/lib/docker/containers/9e1e1f2251252cf45734f00d4548b770d1f775a8e7aafb8e8b3c760d9923276/resolv.conf",
  "HostnamePath": "/var/lib/docker/containers/9e1e1f2251252cf45734f00d4548b770d1f775a8e7aafb8e8b3c760d9923276/hostname",
  "HostsPath": "/var/lib/docker/containers/9e1e1f2251252cf45734f00d4548b770d1f775a8e7aafb8e8b3c760d9923276/hosts",
  "LogPath": "/var/lib/docker/containers/9e1e1f2251252cf45734f00d4548b770d1f775a8e7aafb8e8b3c760d9923276/json-file.log",
  "Name": "/competent_snyder",
  "RestartCount": 0,
  "Driver": "overlay2",
  "Platform": "linux",
  "MountLabel": "",
  "ProcessLabel": "",
  "AppArmorProfile": "docker-default",
  "ExecIDs": null
}
```

```
docker inspect 9e1e1f225125 --format='{{json .}}' | jq
'with_entries(select(.value | type != "object" and type !=
"array"))'
```

Команда: `docker inspect [object_id]`

docker build

Основные инструкции:

FROM – из чего мы собираем наш образ.

RUN – Позволяет выполнить команду во время сборки образа

COPY – Копирует файлы из текущей (если не указано иного) директории внутрь образа

ADD – Аналог COPY* (лучше не использовать)

WORKDIR – Устанавливает рабочую директорию (аналог **cd**, change directory)

ENV – Создает переменные окружения.

EXPOSE – какой порт использует приложение (больше для документации)

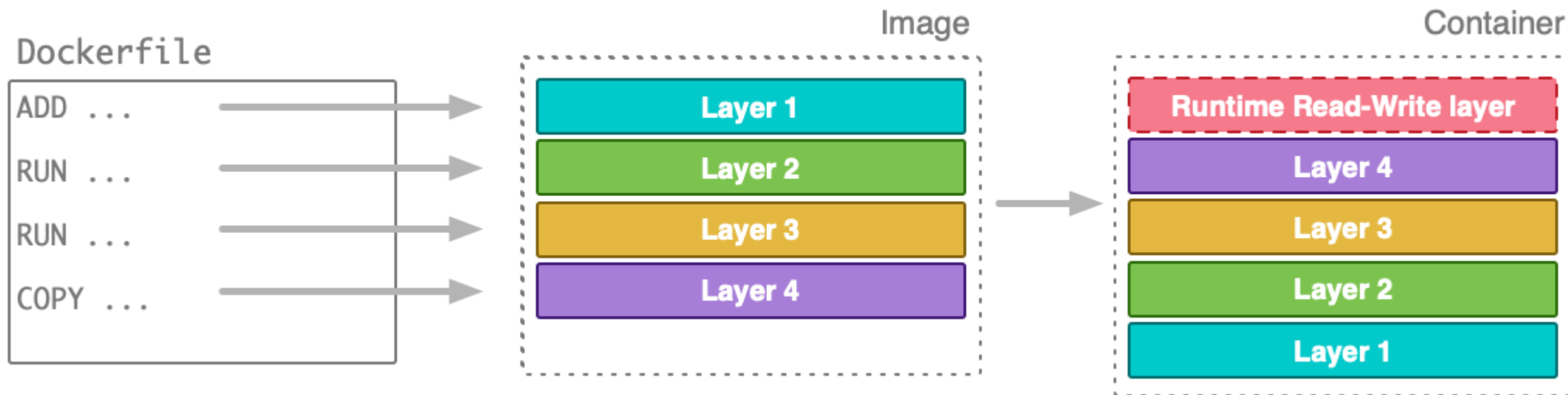
CMD – Определяет команду по умолчанию.

ENTRYPOINT - Определяет основной исполняемый файл контейнера (можно без него)

```
1 FROM php:apache
2
3 RUN ln -s /etc/apache2/mods-available/ssl.load /etc/apache2/mods-enabled/ssl.load
4
5 COPY info.php /var/www/html
6 COPY cert.pem /etc/ssl/certs/ssl-cert-snakeoil.pem
7 COPY key.pem /etc/ssl/private/ssl-cert-snakeoil.key
8
9 RUN rm /etc/apache2/sites-enabled/*
10 RUN rm /etc/apache2/sites-available/*
11 COPY 000-default.conf /etc/apache2/sites-available/default.conf
12 RUN ln -s /etc/apache2/sites-available/default.conf /etc/apache2/sites-enabled/
13
14 # Set the working directory to the web root
15 WORKDIR /var/www/html
16
17
18 EXPOSE 80
19 EXPOSE 443
```

Команда: `docker build -t <image_name>:<tag> -f <docker_file> <build_context>`

docker build (Немного про слои)



Базовый слой — это FROM, то есть исходный образ.

Каждая инструкция создаёт новый слой (layer): FROM, RUN, COPY, ADD и другие. Эти слои накладываются друг на друга и формируют итоговый образ (image)

Ключевая особенность слоёв — это кеширование.

Если изменить любой слой в середине Dockerfile, все последующие слои будут пересобраны заново.

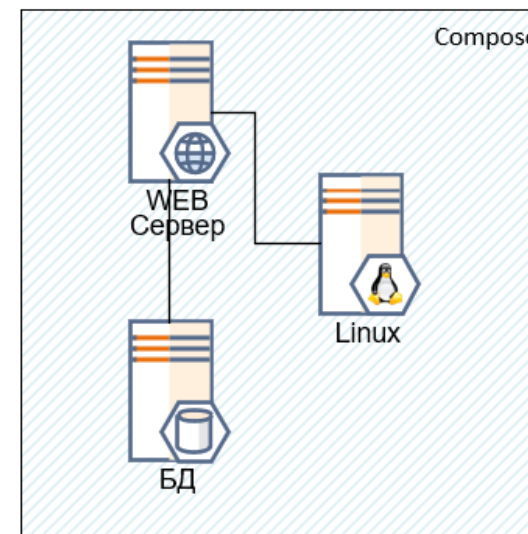
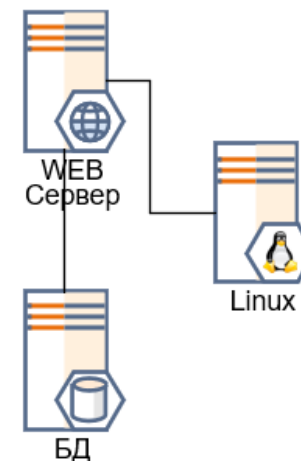
Поэтому **порядок инструкций** в Dockerfile **критически важен**. Лучше использовать оптимизации: писать то, что точно не будет меняться **как можно выше**, а также использовать **склеивание команд** через &&

docker compose

Когда приложений становится много – поможет docker-compose

Если раньше чтобы запустить два+ контейнера, нам надо было вводить две+ команды `docker run`, то с `docker compose` потребуется только сформировать `docker-compose.yml` и выполнить только **одну команду**

Хорошей практикой является минимизация ручных команд и легкая воспроизводимость (повторяемость)



Команда: `docker compose -f <compose_file> up -d`

docker compose

```
1  services:
2    webapp:
3      image: my-webapp
4      networks:
5        - frontend
6        - backend
7
8    db:
9      image: postgres:16
10     networks:
11       - backend
12     volumes:
13       - db_data:/var/lib/postgresql/data
14
15     nginx:
16       image: nginx
17       ports:
18         - "80:80"
19       networks:
20         - frontend
21         - backend
22
23   volumes:
24     db_data:
25
26   networks:
27     frontend:
28       driver: bridge
29
30     backend:
31       driver: bridge
32       internal: true
33
```

Структура следующая

- Services
 - service_name (для Docker DNS):
 - image*
 - networks (Обязательно должен быть определен в глобальном Networks)
 - ports
 - volumes
 - depends_on
 - restart
 - healthcheck
 - env_file
 - environment
- Volumes
 - volume_name
 - Разные параметры
- Networks

Команда: `docker compose -f <compose_file> up -d`



Антон Титов

Руководитель группы защиты веб-приложений